LTIMindtree Technical Interview

Preparation inputs for students who cleared the AI bot interview — what to expect, what to say, and what to write live.

1 · What to expect

Item	Details
Mode & length	Virtual (Teams / Meet), one-on-one with a technical lead or senior engineer · usually 30–45 minutes
Flow	Self-introduction → project deep-dive → core CS (OOPs, DBMS, OS) → 1–2 live programs → SQL queries → a situational question
What they judge	Clear fundamentals, honest ownership of your own project, and how you think aloud — they do not expect perfect answers
Next round	HR (15–20 min) — often scheduled within 1–2 days of the technical round

The golden rule — your introduction sets the agenda

Every skill, language and project you mention in the first 90 seconds becomes a question. Mention only what you can defend for 5 minutes.

If asked to rate yourself, say 6 or 7 out of 10. An 8+ invites deep questions (JVM internals, microservices) that freshers are not expected to know.

Be honest about the bot interview and your resume — interviewers often cross-check what you said earlier.

2 · Self-introduction (60–90 seconds)

#	What to say	Example line
1	Name, degree, branch, college, year	“I'm Priya, a final-year B.E. CSE student at ABC College.”
2	Core skills — 1 language + SQL + 1 framework	“I'm comfortable in Java and SQL, and I've built web apps with React.”
3	One main project in one sentence	“My main project is a hostel complaint tracker with role-based logins.”
4	Internship / certification (if any)	“I completed a 2-month internship on REST APIs at XYZ.”
5	Close with interest in the role	“I'm keen to start my career as an engineer at LTIMindtree.”

Avoid: reading from a script, family details, hobbies unless asked, and listing tools you have only seen in a video.

3 · Project deep-dive — prepare answers to all of these

Area	Questions you will be asked
Purpose	What problem does it solve? Who uses it? How is it different from similar apps?
Your role	Individual or team? What exactly did **you** build? How many members?
Architecture	Explain the data flow from the user's click to the database and back. Draw it if screen-share is allowed.
Tech choices	Why this language / framework / database? What would you replace it with, and why?
Database	Tables and columns, primary and foreign keys, how two entities are related (one-to-many / many-to-many).
Code	Show or explain one important function. Expect “write the login logic” or “write a query from your schema”.
Security	Password hashing, input validation, authentication (sessions / JWT), SQL injection prevention.
Challenges	One real bug or problem and how you fixed it — the most commonly asked project question.
Scale & future	How would it handle 10x users? What feature would you add next?

Homework before the interview

Write 3 SQL queries on your own project's tables without notes, and explain the join in each.

Prepare a 30-second and a 2-minute version of your main project explanation.

4 · Core CS — most asked questions

OOPs

Question	Key points to say
Four pillars	Encapsulation (data + methods, private fields with getters/setters), Inheritance (reuse via parent–child), Polymorphism (one name, many forms), Abstraction (show what, hide how). Give one real example for each.
Overloading vs overriding	Overloading: same name, different parameters, same class, compile-time. Overriding: same signature in child class, run-time.
Abstract class vs interface	Abstract class can have state, constructors and concrete methods; a class extends one. Interface defines a contract; a class implements many.
Constructor types	Default, parameterised, copy (C++). No return type; runs automatically when the object is created. Destructor / garbage collection cleans up.

Question	Key points to say
Shallow vs deep copy	Shallow copies references (inner objects shared). Deep copies the inner objects too (fully independent).
Exception handling	try–catch–finally; checked vs unchecked (Java); throw vs throws; finally always runs. Examples: ArithmeticException, NullPointerException.

Language basics (answer for the language you named)

Question	Key points to say
Java: JDK vs JRE vs JVM	JDK = tools to develop (compiler) + JRE. JRE = JVM + libraries to run. JVM executes bytecode.
Why Java is platform independent	Code compiles to bytecode, which any OS's JVM can run — “write once, run anywhere”.
Java: String immutability, HashMap	Strings cannot change once created (safe, pooled). HashMap stores key–value pairs using hashing; one null key allowed; not synchronised.
Python: list vs tuple vs set	List: ordered, mutable. Tuple: ordered, immutable. Set: unordered, unique items. Dictionary: key–value pairs.
Python: lambda	Anonymous one-line function, e.g. `square = lambda x: x * x`; used with map, filter, sorted.
break vs continue	break exits the loop; continue skips to the next iteration.

DBMS

Question	Key points to say
DBMS vs RDBMS	RDBMS stores data in related tables with keys and enforces integrity (MySQL, PostgreSQL, Oracle).
Keys	Primary (unique + not null), candidate, foreign (links tables), unique (allows one null in most DBs), composite.
Normalization	Removes redundancy. 1NF: atomic values. 2NF: no partial dependency on part of a composite key. 3NF: no transitive dependency. BCNF: every determinant is a candidate key.
ACID	Atomicity (all or nothing), Consistency (valid state), Isolation (transactions don't interfere), Durability (committed data survives failure). Use the bank-transfer example.
SQL vs NoSQL	SQL: fixed schema, tables, joins, ACID. NoSQL (MongoDB): flexible documents, scales horizontally.
Joins, views, triggers	INNER = matching rows; LEFT = all left + matches; view = saved query (virtual table); trigger = runs automatically on INSERT/UPDATE/DELETE.

Operating Systems & Networks

Question	Key points to say
Process vs thread	Process: program in execution with its own memory. Thread: lightweight unit inside a process sharing its memory.
Deadlock	4 conditions: mutual exclusion, hold and wait, no preemption, circular wait. Handle by prevention, avoidance (Banker's algorithm), or detection and recovery.
CPU scheduling	FCFS, SJF, Priority, Round Robin (time quantum) — know one advantage of each.
OSI model	Physical, Data Link, Network, Transport, Session, Presentation, Application — one protocol per layer (IP, TCP, HTTP).
DNS, GET vs POST	DNS converts domain names to IP addresses. GET sends data in the URL (fetch); POST sends it in the body (create / submit).

5 · Live coding — practise until you can write these without help

Most asked: prime check, first N primes, factorial (recursion), Fibonacci, reverse a string **without built-ins**, palindrome, second largest element, count vowels and consonants, power of 2 with exception handling, star / number patterns. Write in your strongest language; say the approach aloud first, then code, then test with 0, 1 and a normal value.

Prime check (√n loop)

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

Power of 2 with exception handling

```
try:
    n = int(input("Enter a number: "))
    if n <= 0:
        raise ValueError("must be positive")
    print(n & (n - 1) == 0)
except ValueError as e:
    print("Invalid input:", e)
```

Reverse a string without built-ins

```
s = input()
rev = ""
for ch in s:
    rev = ch + rev
print(rev)
print("Palindrome" if rev == s else "Not")
```

Second largest (distinct) in one pass

```
first = second = None
for x in arr:
    if first is None or x > first:
        second, first = first, x
    elif x != first and (second is None
                         or x > second):
        second = x
print(second)
```

Factorial using recursion

```
def fact(n):
    if n <= 1:          # base case
        return 1
    return n * fact(n - 1)
```

First n Fibonacci numbers

```
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Java students: be ready to write the same programs with `Scanner`, a class and `main()` — and for exception handling, catch `InputMismatchException` / `NumberFormatException`.

6 · SQL — queries to write live

Tables used: `employees(emp_id, name, salary, dept_id)` and `departments(dept_id, dept_name)`. Interviewers also describe their own small scenario — listen, repeat the columns back, then write.

Second highest salary

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary)
                FROM employees);
```

Nth (e.g. 3rd) highest — DENSE_RANK

```
SELECT DISTINCT salary
FROM (SELECT salary,
        DENSE_RANK() OVER
          (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;
```

Employees per department / more than 5

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id;

-- only departments with more than 5
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

Join + employees without a department

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
  ON e.dept_id = d.dept_id;

SELECT e.name
FROM employees e
LEFT JOIN departments d
  ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Be ready to explain: WHERE vs HAVING, DELETE vs TRUNCATE vs DROP, why DENSE_RANK (handles ties), and SQL injection (never concatenate user input — use parameterised queries / prepared statements).

7 · Resume-based questions — only if it is on your resume

If your resume mentions…	Expect
AI / ML / Data science	What is ML; supervised vs unsupervised; logistic regression and where you'd use it; Type I vs Type II error; NumPy vs Pandas; how AI is used in your project
Web development (MERN / Java FSD)	React components, state and props, hooks used; REST API and HTTP methods; how frontend calls the backend; JWT login flow; MongoDB vs SQL
Cloud / DevOps	IaaS vs PaaS vs SaaS; pros and cons of cloud storage; what Git / GitHub does; basic CI/CD idea
Cybersecurity	Ransomware, phishing, SQL injection, firewall, how to protect users from attacks
Internship / certification	What you did there, tools used, one thing you learned — keep it consistent with what you told the bot

8 · Situational questions & interview conduct

Commonly asked	How to answer
“Client wants delivery before the deadline, but it can't be done.”	Stay honest: explain the risk, propose delivering a working priority part first, and agree a realistic date with the lead.
“Conflict with a team member?”	Use a real example: listened, discussed facts not people, agreed on a plan, focused on the deadline.
“You don't know a technology the project needs?”	Learn the basics quickly (docs, tutorials), build a small sample, ask seniors when stuck.
“Will you relocate / work in shifts?”	Answer clearly and positively — flexibility on location and shifts is checked in both rounds.

9 · Interview day — do's and don'ts

Do

- Join 10 minutes early; test camera, mic, internet; plain background; formal shirt.
- Think aloud while coding — the approach scores even if the code has a bug.
- If you don't know: “I'm not sure, but I think…” and reason from basics.
- Keep a pen and paper ready for dry runs.
- Ask one good question at the end (training, technologies, team).

Don't

- Read answers from another screen — interviewers notice eye movement and pauses.
- Claim skills you can't explain, or rate yourself 9/10.
- Stay silent while stuck — ask for a hint or explain where you are.
- Argue with the interviewer or interrupt their questions.
- Contradict what you told the bot interview or wrote on your resume.

Night-before checklist

✓ 90-second intro rehearsed aloud ✓ Main project explained in 30 s and 2 min ✓ 3 SQL queries on my own schema

✓ 6 programs written once by hand (prime, factorial, reverse, palindrome, second largest, power of 2)

✓ OOPs four pillars with examples ✓ Normalization + ACID ✓ Deadlock + process vs thread ✓ Resume printed and re-read